

What is a Neural Network?

Definition

A **neural network** is a computational system designed to **simulate the way the human brain learns and processes information**. It consists of layers of **interconnected nodes (neurons)** that collectively learn to represent complex patterns in data.

Mathematically, a neural network is a function $f(X, W)$ that maps input features X to an output Y through a series of **weighted transformations** and **non-linear activations**.

In simple terms

Neural networks learn from data by adjusting internal parameters (weights and biases) so that their predictions get closer to actual outcomes.

Motivation

In traditional machine learning models such as linear regression or decision trees, the relationship between input and output must often be **explicitly defined or engineered**.

However, neural networks **automatically discover representations**. They learn *how* to represent complex relationships directly from data.

This makes them powerful for tasks such as the following:

- Image and speech recognition
- Natural language processing (NLP)
- Machine translation
- Predictive analytics

Core Components

Component	Description	Example
Neuron (Node)	The basic computational unit. Each neuron performs a weighted sum of inputs, followed by an activation function	$z = \sum x_i w_i + b$

Weights (W)	Numerical parameters that determine the importance of each input feature	High weight → Stronger influence
Bias (b)	A constant that allows the activation threshold to shift	Helps fit data not passing through the origin
Activation function (f)	Introduces non-linearity, allowing networks to model complex relationships	ReLU, Sigmoid and Tanh
Layers	Arrangements of neurons; includes input, hidden and output layers	Multilayer feedforward design

Analogy

Think of a neural network as a layered decision process:

- The **input layer** receives raw data.
- The **hidden layers** progressively learn abstract features (such as edges, shapes or word patterns).
- The **output layer** produces final predictions (such as a class label or score).

Example of an NLP task:

'Riya loves NLP.' → Input (word vectors) → Hidden layers (context understanding) → Output ('Positive Sentiment')

Mathematical Perspective

For a single neuron:

$$y = f\left(\sum_{i=1}^n x_i w_i + b\right)$$

For a full network:

$$y' = f_2(W_2f_1(W_1x + b_1) + b_2)$$

Each layer’s output becomes the input for the next layer, forming a **compositional hierarchy** of functions.

Why Neural Networks Are Powerful

1. Learn Non-Linear Relationships

They capture complex mappings between the input and output.

2. Automatic Feature Learning

There is no need for handcrafted features; the model learns what matters.

3. Generalisation

It can adapt to a wide variety of data types and patterns.

4. Scalability

The performance improves with larger data sets and deeper architectures.

Applications of Semantic Processing

- **Search engines:** Understand the intent behind queries
- **Chatbots/Virtual assistants:** Interpret natural user commands
- **Machine translation:** Preserve meaning across languages
- **Information retrieval:** Match queries with conceptually similar documents
- **Sentiment analysis:** Understand underlying emotions and attitudes
- **Question answering systems:** Extract correct factual answers from text

Limitations

Aspect	Challenge
Data hungry	Requires large volumes of training data

Computationally expensive	Needs GPUs or high-performance hardware
Black box nature	Difficult to interpret inner workings
Overfitting	Can memorise training data if not regularised

Example: Simple Neural Network Prediction

Suppose we want to predict student performance based on study hours (x).

Each neuron performs:

$$z = w \cdot x + b$$

$$y' = f(z)$$

If f = sigmoid, output y' lies between 0 and 1, representing probability of success.

Through training, the network **adjusts weights** w and bias b to minimise error.

Comparison With Human Neuron

Definition and Analogy

A **neural network** draws its conceptual inspiration from how the **human brain** processes information through interconnected **neurons**.

In the brain, neurons communicate through electrical impulses; in artificial networks, neurons communicate through **mathematical signals** and **weighted connections**.

Biological Neuron: Structure

Part	Function
Dendrites	Receive input signals from other neurons
Cell body (Soma)	Processes the incoming signals and decides whether to activate
Axon	Transmits output signals to other neurons
Synapse	Connection point between neurons; the signal strength can increase or decrease with learning

Artificial Neuron: Structure

Component	Mathematical Equivalent	Purpose
Inputs	$x_1, x_2, x_3, \dots, x_n$	Represent data features (e.g., pixels, words, and values)
Weights	$w_1, w_2, w_3, \dots, w_n$	Represent the strength of connection between neurons
Summation function	$z = \sum w_i x_i + b$	Aggregates weighted inputs
Activation function	$a = f(z)$	Decides whether the neuron should 'fire'
Output	a	Passed to the next layer of neurons

Visual Comparison

Human Neuron	Artificial Neuron
Receives electrical signals via dendrites	Receives numerical inputs (features)
Processes signal in soma (cell body)	Computes the weighted sum of inputs
Fires a signal through axon if the threshold is reached	Activates output via an activation function
Learns by changing the synaptic strength	Learns by updating weights (via backpropagation)

Information Flow

In a Biological Neuron

1. The input impulses arrive at **dendrites**.
2. Signals are integrated in the **cell body**.
3. If the signal strength crosses a **threshold**, the neuron fires.
4. The signal travels down the **axon** to other neurons via **synapses**.

In an Artificial Neuron

1. Inputs (x_i) are multiplied by **weights** (w_i).
2. All weighted inputs are **summed**, and a **bias** is added:

$$z = \sum w_i x_i + b$$

3. This sum passes through an **activation function** to produce output $a = f(z)$.
4. The output is forwarded to the neurons in the next layer.

Learning Mechanisms: Comparison

Human Brain	Artificial Neural Network
Learns through experience and reinforcement ; synapses strengthen with repeated exposure	Learns through training ; weights are updated to minimise error (loss function)
Adapts gradually to new patterns using neuroplasticity	Adapts through iterative optimisation (e.g., gradient descent)
Learning is unsupervised and generalised	Learning is typically supervised (with labelled data)

Mathematical Representation

Each artificial neuron performs:

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

where:

- x_i : Input features
- w_i : Connection weights
 - b : Bias term
- $f(\cdot)$: Activation function controlling output

If f acts like a biological threshold (e.g., sigmoid function), it determines whether the neuron 'fires' (activates).

Neural Network: Architecture

Definition

The **architecture of a neural network** defines how its neurons are structured and connected.

It determines **how data flows** from inputs to outputs and how learning occurs each stage.

At its core, a neural network is organised into **layers of neurons**, where each neuron performs a mathematical operation on the incoming data and passes the result forward.

These layers, connections and flow of activations form the **computational structure** or architecture of the network.

Architecture: Basic Components

Component	Description	Role in Learning
Input layer	Receives raw data (features)	Passes input features to the next layer without computation
Hidden layers	Perform weighted computations and activations	Extract features and learn representations
Output layer	Produces the final prediction or probability	Generates task-specific output (e.g., class probabilities)
Connections (Weights)	Link neurons across layers	Determine the strength of influence of one neuron on another
Bias terms	Additional constants added to activations	Allow flexible shifting of activation thresholds

Typical Feedforward Architecture

A **feedforward neural network (FNN)** or **multilayer perceptron (MLP)** is the simplest form of architecture.

Information moves **in one direction only** – from input to output – without cycles or feedback.

Example Structure

Input Layer → Hidden Layer 1 → Hidden Layer 2 → Output Layer

Each neuron in one layer connects to **every neuron** in the next layer. This is called a **fully connected (dense) network**.

Mathematical Flow

For a given layer l :

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

where:

- $a^{(l-1)}$: Activations from the previous layer
 - $W^{(l)}$: Weight matrix of layer l
 - $b^{(l)}$: Bias vector
 - $f(\cdot)$: Activation function
 - $a^{(l)}$: Output of layer l

Information Flow

1. Input Stage

Raw features (e.g., image pixels, words and sensor readings) enter through the input layer.

2. Weighted Transformation

Each neuron computes a weighted sum of its inputs plus bias.

3. Activation

The activation function decides the output signal strength for that neuron.

4. Propagation

The activated outputs become inputs for the next layer.

5. Output Stage

The final layer transforms last activations into predictions (e.g., probabilities and classifications).

Network Architectures: Types

Type	Structure	Key Feature	Use Case
Feedforward Network (FNN)	Layers connected sequentially; no feedback	Unidirectional flow of data	Classification and regression
Convolutional Neural Network (CNN)	Uses filters/kernels over spatial data	Captures local spatial patterns	Image, video and vision tasks
Recurrent Neural Network (RNN)	Loops over sequences	Maintains memory of past inputs	Text, speech and sequential data

(Subsequent modules explore CNNs and RNNs in depth.)

Example: 3-Layer Neural Network

Layer	Input/Output Size	Computation
Input Layer	3 input features	—

Hidden Layer	4 neurons	$z_1 = W_1x + b_1; a_1 = f(z_1)$
Output Layer	1 neuron	$y' = f(w_2a_1 + b_2)$

Flow

Input → Weighted Sum → Activation → Output

Hyperparameters Defining Architecture

Hyperparameter	Purpose
Number of layers	Determines the network depth and learning capacity
Number of neurons per layer	Controls the representational power
Activation function	Governs non-linearity in transformations
Initialisation strategy	Influences convergence during training
Regularisation (dropout, L2)	Prevents overfitting by constraining weight growth

Architectural Intuition

- **Shallow networks** learn simple mappings but struggle with complex patterns.
- **Deep networks** stack multiple layers, enabling hierarchical learning, from low-level features to abstract concepts.

Example:

In an image model:

- Early layers learn edges and textures.
- Mid layers learn shapes or parts.
- Deeper layers learn complete objects.

Deep Architectures: Advantages

Aspect	Benefit
Hierarchical learning	Captures simple and complex patterns
Generalisation power	Adapts to diverse input types
Automatic feature extraction	Reduces the need for manual engineering

Activation Function: Role

Definition

An **activation function** is a mathematical operation applied to the output of each neuron in a neural network to introduce **non-linearity**.

It determines whether a neuron should be **activated (fire)** based on the strength of its input signal.

Without activation functions, the entire neural network would behave similar to a **linear regression model** regardless of the number of layers it has, severely limiting its ability to learn complex relationships.

In essence, the activation function provides the network its *intelligence*, allowing it to learn non-linear decision boundaries.

Mathematical Context

Each neuron performs:

$$z = \sum_{i=1}^n w_i x_i + b$$

The activation function $f(z)$ transforms this linear combination into a non-linear output:

$$a = f(z)$$

Here, a is the neuron’s final activation (output).

This transformation enables the network to approximate any arbitrary function.

Need for Activation Functions

Reason	Explanation
Non-linearity	Real-world data is complex and non-linear. Non-linear activations let the network model such relationships.
Learning complex features	Layers can combine and transform data into higher-level abstractions.
Information flow control	Activation decides whether a neuron 'fires', controlling the signal passed forward.
Universal approximation	A network with non-linear activations can approximate any continuous function.

Common Activation Functions

Function	Mathematical Formulation	Range	Remarks
----------	--------------------------	-------	---------

Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	$(0, 1)$	Smooth; maps values to the probability range. Common in output layers for binary classification
Tanh	$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$(-1, 1)$	Zero-centred version of Sigmoid; better gradient flow
Rectified linear unit (ReLU)	$f(x) = \max(0, x)$	$[0, \infty)$	Fast convergence; most widely used for hidden layers
Leaky ReLU	$f(x) = x \text{ if } x > 0, 0.01x \text{ otherwise}$	$(-\infty, \infty)$	Prevents neuron inactivity ('dying ReLU' problem)
Exponential linear unit (ELU)	$f(x) = x \text{ if } x > 0, \alpha(e^x - 1) \text{ if } x \leq 0$	$(-\alpha, \infty)$	Smooths ReLU behaviour for negative values
Softmax	$f_{i(x)} = \frac{e^{x_i}}{\sum_j e^{x_j}}$	$(0, 1)$ per class	Converts logits into probabilities in multiclass outputs

Graphical Intuition

- **Sigmoid:** S-shaped curve, compresses inputs to $[0,1]$
- **Tanh:** Similar to sigmoid but symmetric around 0
- **ReLU:** Straight line for positive inputs, flat for negative inputs
- **Softmax:** Produces normalised probability distribution across multiple outputs

Example: Non-Linearity in Action

Imagine classifying points on a 2D plane.

- Without Activation

The model can only separate data with a straight line (linear decision boundary).

- With Activation (e.g., ReLU or Tanh)

The model can curve and bend the boundary to correctly separate complex clusters.

Thus, activations allow neural networks to model phenomena such as the following:

- Speech patterns
- Facial features
- Sentiment in text
- Non-linear financial or medical data

Activation by Layer Type

Layer Type	Common Activation Function	Reason
Hidden layers	ReLU, Leaky ReLU and Tanh	Efficient and handle gradient propagation well
Output (binary classification)	Sigmoid	Produces a single probability (0–1)
Output (multiclass classification)	Softmax	Produces class probabilities summing to 1
Output (regression)	Linear	Direct numerical output without squashing

Hidden Layers: Role

Definition

Hidden layers are the layers in a neural network that lie **between** the input and output layers.

They are called 'hidden' because we do not directly observe their values; instead, they internally transform the input data into increasingly meaningful representations.

Hidden layers are where the learning happens.

They extract patterns, capture relationships and build hierarchical features from raw data.

Purpose of Hidden Layers

Hidden layers enable the network to:

1. Learn Complex Patterns

Instead of mapping input → output directly, hidden layers allow multistep transformations.

2. Extract Intermediate Representations

Each layer creates more abstract features than the previous one.

3. Model Non-Linear Relationships

Combined with activation functions, hidden layers allow the network to capture non-linear patterns in data.

4. Build Hierarchical Understanding

Early layers learn simple features; deeper layers learn high-level concepts.

Why a Single Layer Is Not Sufficient

A neural network **without hidden layers** is equivalent to the following:

- Linear regression (for continuous output)
- Logistic regression (for binary classification)

Such a network can learn only **straight-line relationships**.

However, tasks such as the following require modelling complex, non-linear dependencies:

- Image recognition
- Speech processing
- Sentiment analysis
- Language translation

Hence, **multiple hidden layers** (deep networks) outperform shallow models.

How Hidden Layers Transform Data

Each hidden layer applies the following:

1. Weighted Transformation

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

2. Activation Function

$$a^{(l)} = f(z^{(l)})$$

These steps **reshape** data, emphasising useful information, suppressing noise and identifying task-relevant features.

Example: From Raw Data to Concepts

Consider an image of a handwritten digit '5'.

Layer	What It Learns
Input	Raw pixel values
Hidden Layer 1	Edges and corners

Hidden Layer 2	Curves and strokes
Hidden Layer 3	Entire digit pattern
Output	Predicted digit = 5

Even in NLP:

For example, for the sentence 'The movie was absolutely wonderful.'

Layer	Learning
Hidden 1	Word-level patterns (wonderful → positive)
Hidden 2	Phrase-level meaning ('absolutely wonderful')
Hidden 3	Overall sentiment context
Output	Positive sentiment

Depth and Expressive Power

Shallow Networks

- Fewer hidden layers deep networks
- Can only learn simple decision boundaries
- Lower expressive power than deep networks

Deep Networks

- More hidden layers
- Can approximate highly complex functions
- Hierarchical feature extraction improves generalisation

Universal Approximation Theorem

A neural network with at least **one hidden layer** and non-linear activation can approximate any continuous function.

However, deeper networks are **more efficient at this task**.

Choosing the Number of Hidden Layers

Task Complexity	Typical Depth
Simple tabular data	1–2 hidden layers
Image recognition	10–100+ layers
NLP (Transformers)	12–96 layers
Speech processing	5–20 layers

In classical MLPs, 1–3 hidden layers are common.

Effect of Hidden Layer Size

Hidden Layer Size	Impact
Too small	Underfitting; cannot capture patterns
Too large	Overfitting; memorises training data
Balanced	Good generalisation

Example: Hidden Layer Transformation (Simple Numeric)

Input Vector

$$x = [2, 3]$$

Suppose a hidden layer has 3 neurons.

Weights:

$$W = \begin{bmatrix} 0.2 & 0.5 \\ 0.1 & 0.4 \\ 0.7 & -0.3 \end{bmatrix}$$

Bias:

$$b = [1, 0, -1]$$

Compute:

$$z = Wx + b = \begin{bmatrix} 2.1 \\ 1.4 \\ 0.5 \end{bmatrix}$$

After activation (ReLU):

$$a = [2.1, 1.4, 0.5]$$

The hidden layer creates **new features** that are not directly present in the input.

Advantages of Hidden Layers

Aspect	Benefit
Feature learning	Automatically discovers patterns
Non-linear modelling	Captures complex relationships
Hierarchical representations	Builds knowledge step-by-step
Generalisation	Learns meaningful abstractions

Forward Propagation

Definition

Forward propagation (forward pass) is the process through which input data travels **from the input layer through all hidden layers to the output layer** to produce a prediction.

It is the **inference phase** of a neural network, where computations occur layer by layer.

In simple terms, forward propagation computes the output of a network for a given input by applying weights, biases and activation functions.

Purpose of Forward Propagation

1. **Compute predictions** for the current input
2. **Measure how incorrect the model is** using a loss function
3. **Provide values for backpropagation** to compute gradients

Forward pass is the first half of the training loop (the second half is backward pass).

How Forward Propagation Works

For each layer:

1. Weighted Sum

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

2. Activation

$$a^{(l)} = f(z^{(l)})$$

Where:

- $W^{(l)}$: Weight matrix of layer l
- $b^{(l)}$: Bias vector
- $a^{(l-1)}$: Activations from previous layer
- f : Activation function (ReLU, Sigmoid, etc.)

The final output layer produces the prediction y' .

Computational Flow Diagram

Input (x)



Weighted sum ($z = Wx + b$)



Activation ($a = f(z)$)



Weighted sum (next layer)



Activation



Output ($\hat{y}$)

Every neuron repeats the same two operations:

1. Linear transformation
2. Non-linear activation

Types of Outputs Depending on Tasks

Task	Final Activation	Output Interpretation
Binary classification	Sigmoid	Probability (0–1)
Multiclass classification	Softmax	Class probabilities
Regression	Linear (no activation)	Continuous numeric value

Loss Function

After the forward pass, the model computes the **loss**, measuring how inaccurate the predictions are.

Examples:

- Binary cross-entropy for binary classification
- Categorical cross-entropy for multiclass classification
- Mean squared error for regression

Loss is crucial for **backpropagation**, which is the next step.

Why Forward Propagation Matters

Aspect	Importance
Prediction	Converts input into output
Learning	Produces the activations needed for gradients
Evaluation	Determines the model performance on training and test data
Building blocks	Form the basis for convolutional networks, RNNs, transformers, etc.

Backward Propagation

Definition

Backward propagation, commonly called **backpropagation**, is the algorithm used to **train neural networks** by adjusting their weights and biases.

It computes how much each weight contributed to the overall prediction error and updates the weights in the **opposite direction of the gradient** to reduce future errors.

In simple terms:

Forward pass → Compute the prediction

Backward pass → Compute the error and correct the network

Backpropagation is **at the core of learning** in neural networks.

Purpose of Backpropagation

1. **Calculate the gradients** of the loss with respect to each weight and bias.
2. **Update weights** to reduce the prediction error.
3. Enable the network to **learn patterns from data** over many iterations (epochs).

Key Idea: Chain Rule of Calculus

Backpropagation relies on the **chain rule** to compute the derivatives of the loss through multiple layers.

If:

$$y' = f(W_2 f(W_1 x + b_1) + b_2)$$

Then:

$$\frac{\partial L}{\partial W_2}, \quad \frac{\partial L}{\partial W_1}$$

are computed by chaining gradients from the following:

- Loss function
- Output layer
- Hidden layers
- Input features

Three Steps in Backpropagation

1. Compute the Loss

Compare the predicted output $\hat{y}$ with the true label y .

Example (for regression)

$$L = \frac{1}{2}(y' - y)^2$$

2. Compute Gradients (Layer by Layer)

For each weight:

$$Gradient = \frac{\partial L}{\partial W}$$

This indicates **how changing that weight affects the loss**.

Gradients are computed **from the output layer backward**, hence the name.

For the output layer:

$$\delta^{(2)} = (y' - y) \cdot f'(z^{(2)})$$

For the hidden layer:

$$\delta^{(1)} = (W^{(2)})^T \delta^{(2)} \cdot f'(z^{(1)})$$

3. Update Weights

Weights are updated using gradient descent:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}$$

Where:

- η = Learning rate
- The gradient defines the direction of change
- Subtraction because we want to **minimise** loss

Intuitive Analogy

Backprop is similar to learning to play basketball:

1. You shoot (forward pass).
2. See how far you missed (loss).
3. Adjust your aim based on the error (backprop update).
4. Try again; repeat many times (epochs).

Detailed Numerical Example (Tiny Network)

Given:

Input:

$$x = [1, 2]$$

True output:

$$y = 1$$

Weights:

$$W^{(1)} = [0.3, 0.4] \quad b^{(1)} = 0.1$$

$$W^{(2)} = [0.6] \quad b^{(2)} = -0.2$$

Activation: Sigmoid

Forward Pass

1. Hidden layer:

$$z^{(1)} = 0.3(1) + 0.4(2) + 0.1 = 1.2$$

$$a^{(1)} = \sigma(1.2) = 0.768$$

2. Output layer:

$$z^{(2)} = 0.6(0.768) - 0.2 = 0.2608$$

$$y' = \sigma(0.2608) = 0.5648$$

Loss (MSE)

$$L = \frac{1}{2(0.5648 - 1)^2} = 0.0945$$

Backward Pass

Output Layer Gradient

$$\delta^{(2)} = (y' - y) \cdot \sigma'(z^{(2)})$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\sigma'(0.2608) = 0.2456$$

$$\delta^{(2)} = (0.5648 - 1) \cdot 0.2456 = -0.107$$

Gradient for $W^{(2)}$:

$$\frac{\partial L}{\partial W^{(2)}} = a^{(1)} \cdot \delta^{(2)} = 0.768(-0.107) = -0.082$$

Hidden layer gradient:

$$\begin{aligned}\delta^{(1)} &= W^{(2)}\delta^{(2)} \cdot \sigma'(1.2) \\ &= 0.6(-0.107) \cdot 0.177 = -0.0113\end{aligned}$$

Gradient for $W^{(1)}$:

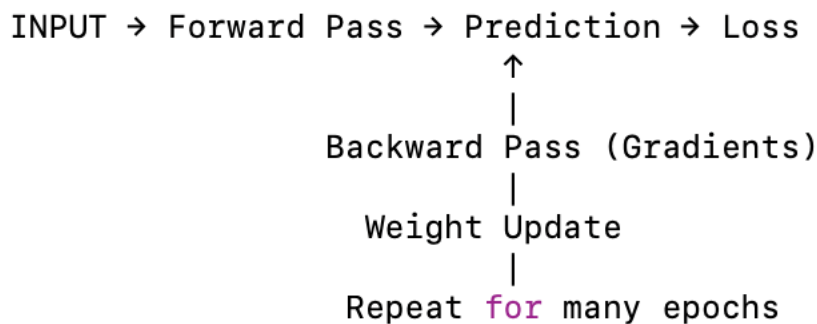
$$\frac{\partial L}{\partial W^{(1)}} = x \cdot \delta^{(1)} = 1, 2 = [-0.0113, -0.0226]$$

Weight Update (Learning Rate = 0.1)

$$\begin{aligned}W_{new}^{(1)} &= W^{(1)} - 0.1[-0.0113, -0.0226] \\ W_{new}^{(2)} &= 0.6 - 0.1(-0.082) = 0.6082\end{aligned}$$

Weights increase or decrease depending on the sign of the gradient.

Visualisation of the Complete Training Loop



Why Backpropagation Is Essential

Function	Importance
----------	------------

Learns from mistakes	Adjusts weights so the next prediction improves
Efficient	Avoids brute-force trial and error
Scalable	Works for deep networks with millions of parameters
Foundational	Basis of training CNNs, RNNs, transformers, etc.

Common Issues with Backpropagation

Issue	Cause	Effect
Vanishing gradients	Sigmoid/tanh in deep nets	Slow or no learning
Exploding gradients	Unstable weight updates	Divergence
Local minima / saddle points	Complex loss surface	Model gets stuck
Overfitting	Too many parameters	Poor generalisation

Solutions include the following:

- ReLU activation
- Batch normalisation
- Dropout
- Optimisers such as Adam
- Gradient clipping

Types of Optimisers

Definition

An **optimiser** is an algorithm used to **update the weights and biases** of a neural network to minimise the loss function.

During training, optimisers use the gradients computed from backpropagation to determine **how much and in what direction** each parameter should change.

In simple terms, the optimiser decides how the neural network learns by adjusting weights efficiently.

Need for Optimizers

- Gradients only indicate **the direction** to move in.
- Optimisers decide **how big** each step should be.
- They help avoid problems such as **slow learning**, **oscillations** or **getting stuck** in poor local minima.

Efficient optimisation techniques are crucial to training deep networks effectively.

Key Concepts in Optimisation

1. Learning Rate (η)

It controls the step size during weight updates.

$$W_{new} = W_{old} - \eta \cdot \frac{\partial L}{\partial W}$$

Too high → Divergence

Too low → Very slow training

2. Gradient

The slope of the loss function shows the direction to minimise.

3. Momentum

It smooths updates by remembering previous gradients.

4. Adaptive Learning Rate

It adjusts the learning rate for each parameter individually based on the gradient history.

Types of Optimisers

1. Gradient Descent (Batch Gradient Descent)

How it works

- Computes the gradient using the **entire data set**
- Updates weights **once per epoch**

Update Rule

$$W := W - \eta \nabla_w L$$

Pros

- Stable updates
- Suitable for convex problems

Cons

- Very slow for large data sets

- Requires vast memory

2. Stochastic Gradient Descent (SGD)

How It Works

- Updates the weights **for each training example**

Update Rule

$$W := W - \eta \nabla_w L(x_i)$$

Pros

- Fast updates
- Good generalisation

Cons

- Very noisy updates
- Loss fluctuates

3. Mini-Batch Gradient Descent

This is the most commonly used one.

How It Works

- Uses a **small batch (e.g., 32, 64, 128)**
- Combines the speed of SGD with the stability of batch GD

Pros

- Efficient
- Smooth convergence
- Parallelisable on GPUs

4. Momentum Optimiser

This adds a fraction of the previous update to the current update

Update Rule

$$v_t = \gamma v_{t-1} + \eta \nabla_W L$$

$$W := W - y_t$$

Where γ (momentum term) is usually 0.9

Intuition

It helps accelerate learning in the right direction and dampen oscillations.

5. Nesterov Accelerated Gradient (NAG)

It is a better version of momentum that evaluates the gradient at a predicted future position before updating.

Update Rule (Conceptually)

- Calculate the gradient at a **future position** of weights
- Adjust the update accordingly

Advantage

It is characterised by faster convergence and is more stable than the standard momentum.

6. RMSProp

This uses a **moving average of the squared gradients** to adapt the learning rate.

Update Rule

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

$$W := W - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

Pros

- Works well for **non-stationary** problems
- Is excellent for RNNs

7. Adaptive Moment Estimation (ADAM)

It is the **most widely used optimiser** in deep learning.

Combines the following:

- Momentum (first moment)
- RMSProp (second moment)

Update Rule (Simplified)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$W := W - \frac{\eta m_t}{\sqrt{v_t} + \epsilon}$$

Pros

- Fast convergence
- Works well on sparse gradients
- Robust for large data sets and deep networks

Default Parameters

- $\beta_1 = 0.9$
- $\beta_2 = 0.999$
- $\epsilon = 10^{-8}$